Rectangle Partition Coverage

Approaches to Rectangle Coverage Optimization

Gonçalo Branquinho
Gustavo dos Santos
André Sousa

Chapter 1

Introduction

In this report, a geometric coverage problem is presented and solved using various combinatorial optimization techniques. The problem consists of a set of rectangles and their respective vertices. The primary objective is to deploy a set of guards strictly on these vertices such that every rectangle is covered. A rectangle is considered covered if at least one of its incident vertices possesses a guard. Consequently, the initial problem is to determine a vertex assignment that minimizes the total number of guards required.

Subsequently, an extension to this baseline problem is explored. In practical contexts, guards may represent resources that require differentiation, preventing interference or scheduling conflicts. Therefore, a secondary objective is introduced, which consists of assigning a color to each placed guard such that no rectangle is covered by more than one guard of the same color, while minimizing the total number of colours utilized. This formulation effectively transforms the problem into a multi-objective optimization problem.

The following chapters establish mathematical formulations using Greedy Approaches, Integer Programming, and Constraint Programming. Then, the implementation of these approaches using Google OR-Tools and an independent search method maintaining arc-consistency with AC-3 is described. Finally, the multi-objective extension is formulated and solved using a sequential optimization method.

1.1 Objectives

The primary objectives of this report are defined as follows:

1. To evaluate Greedy Approaches as a preliminary method for the guard placement problem.
2. To establish rigorous mathematical formulations for the problem utilizing both Integer Programming and Constraint Programming models.
3. To implement the proposed mathematical models utilizing the Google OR-Tools suite.
4. To develop and evaluate a custom search algorithm employing AC-3 for information filtering and backtracking to solve the Constraint Optimization Problem.
5. To formulate and solve a multi-objective extension of the problem, focusing on the minimization of the colours assigned to the minimum number of guards found in the initial models.

Chapter 2

Greedy Approaches

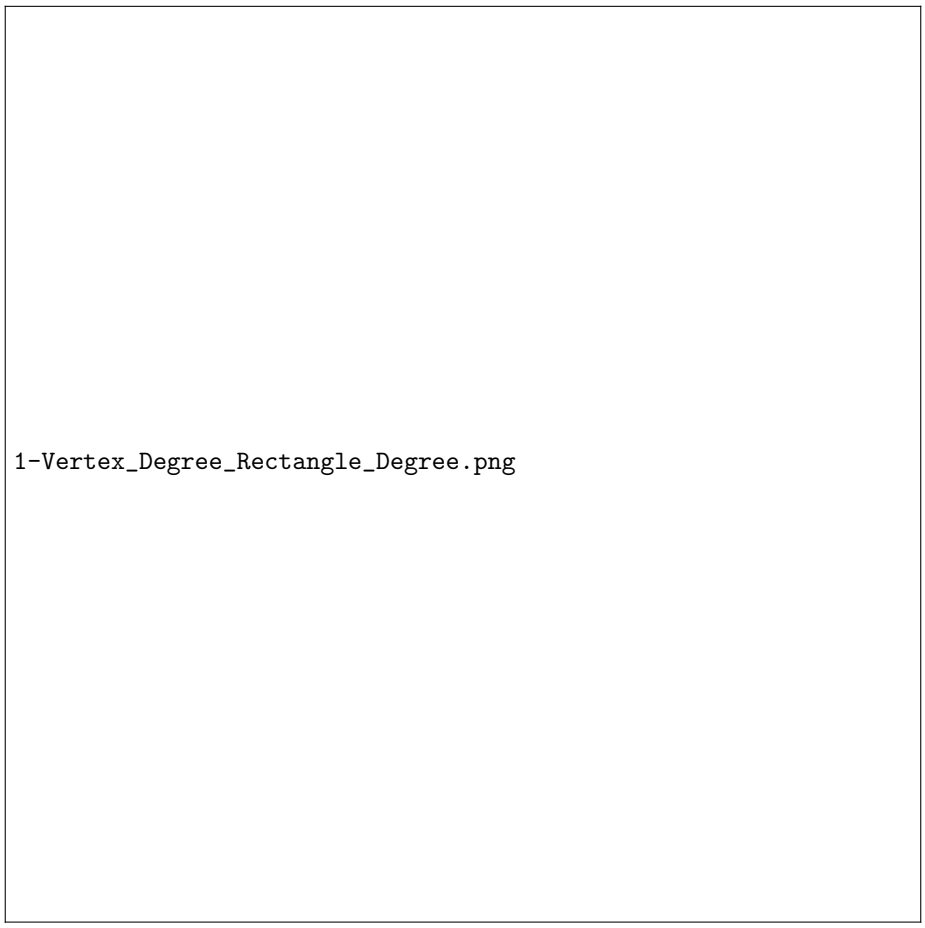
In this chapter, we present all of our greedy approaches and provide a comparative analysis of their performance both among themselves and against the optimal solution.

2.1 Approaches

Before presenting the approaches, it is necessary to introduce two key underlying concepts shared by most of our greedy algorithms: vertex degree and rectangle degree updating. Note that rectangle degree updating is only used in **Greedy3.cpp**, **Greedy3v2.cpp**, and **Greedy4.cpp**.

Regarding vertex degree, it is simply defined as the number of rectangles to which a vertex is incident.

Regarding vertex and rectangle degree update, considering the following image of a random subset of rectangles of an entire a partition (orange borders), supposing the green vertex is selected, then the rectangles that vertex is incident to will be covered (green as well). With that being said, only the vertices and rectangles that are in direct contact with the now covered green rectangles (including the selected vertex) will need to be correctly updated, that is, have their degre decremented (all displayed as blue).



Greedy1.cpp: Our first approach consists of iteratively placing guards on the most impactful vertices, i.e., those that cover the largest number of rectangles, with the goal of minimizing the total number of guards required to cover the entire partition. To achieve this, during input processing, we associate a degree with each vertex, corresponding to the number of rectangles to which it is incident ($1 \leq \text{degree} \leq 3$). After computing all vertex degrees, we insert them into a priority queue and repeatedly extract the top element, which corresponds to the vertex covering the most rectangles (breaking ties using a custom comparator based on vertex coordinates). Each extracted vertex is then assigned a guard and the adjacent vertices are updated accordingly. This process is repeated until all rectangles are covered.

Greedy2.cpp: Our second approach is the inverse of the first one. Instead of iteratively placing guards on vertices with the highest degree extracted from a priority queue, we begin by assuming that all vertices have a guard assigned

initially and then iteratively remove guards from vertices with the lowest degree. This process continues until no further guard can be removed without leaving at least one rectangle uncovered.

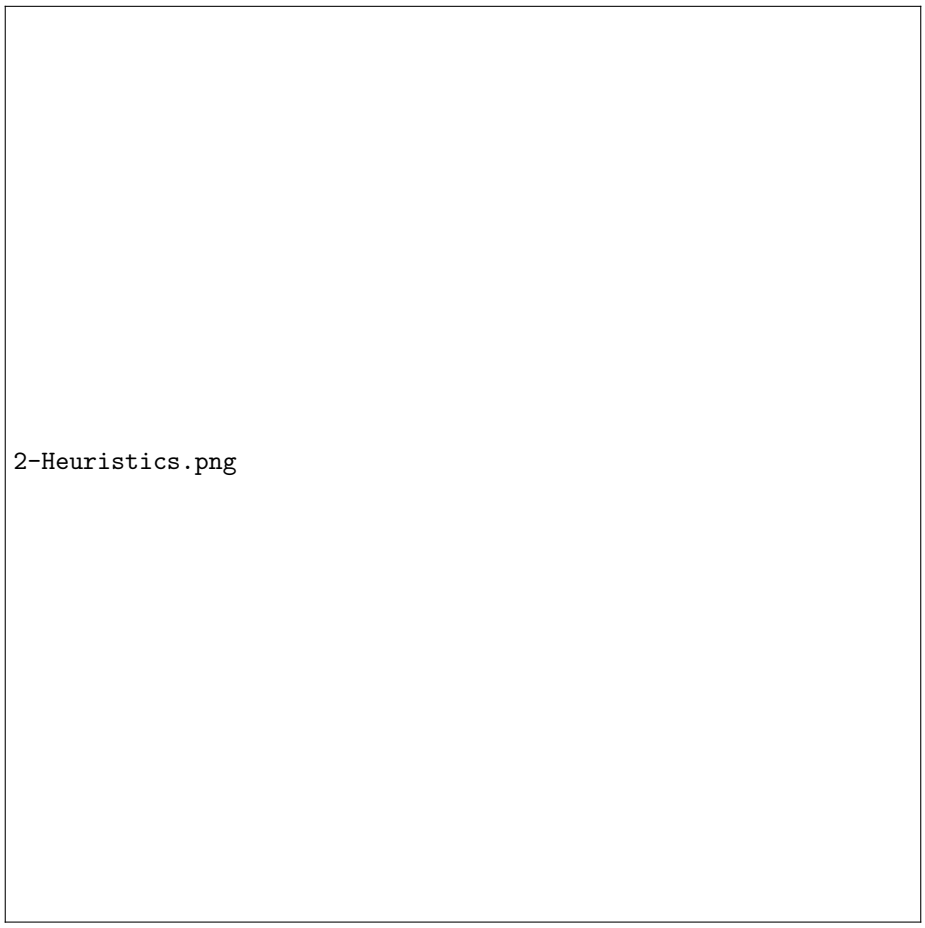
Our subsequent approach, and the two that followed, were based on observations from the first algorithm. We noticed that certain "isolated" rectangles tended to be covered later in the process, which often led to poor and inefficient guard placement decisions. To address this, we shifted the perspective from vertex-based degree prioritization to rectangle-based degree prioritization.

Intuitively, less "isolated" rectangles were more likely to be covered through well-chosen vertices, as they either shared more vertices or were incident to higher-degree vertices. In contrast, more "isolated" rectangles were less flexible and therefore more likely to constrain future decisions if not handled early. This motivated prioritizing lower-degree rectangles earlier in the selection process.

The concept of "isolation" is directly related to a rectangle's degree and may vary depending on the heuristic being applied. In this context, the degree of a rectangle can be defined in one of the following ways:

1. The sum of the degrees of its incident **shared** vertices, i.e., vertices whose degree $\neq 1$.
2. The number of **distinct** rectangles with which it shares vertices.

Regardless of the approach, once the most isolated rectangle is selected, we aim to maximize coverage by choosing its incident vertex with the highest degree. In case of a tie, the rectangle with the lowest identifier is chosen.

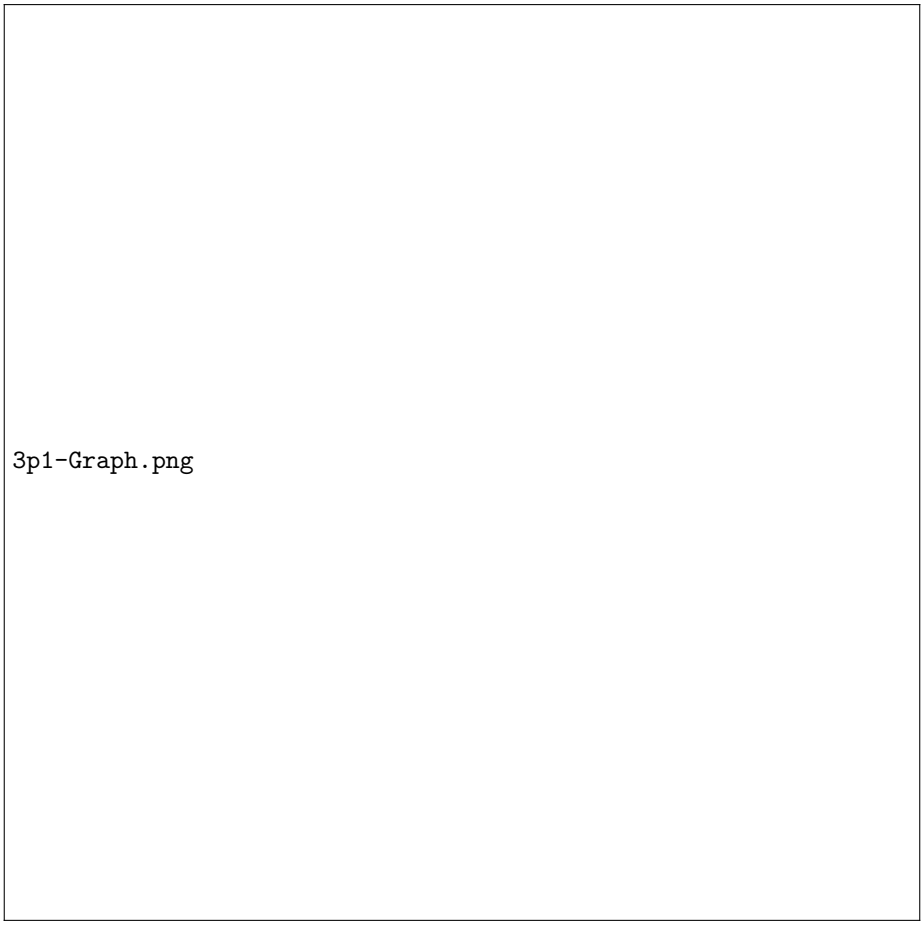


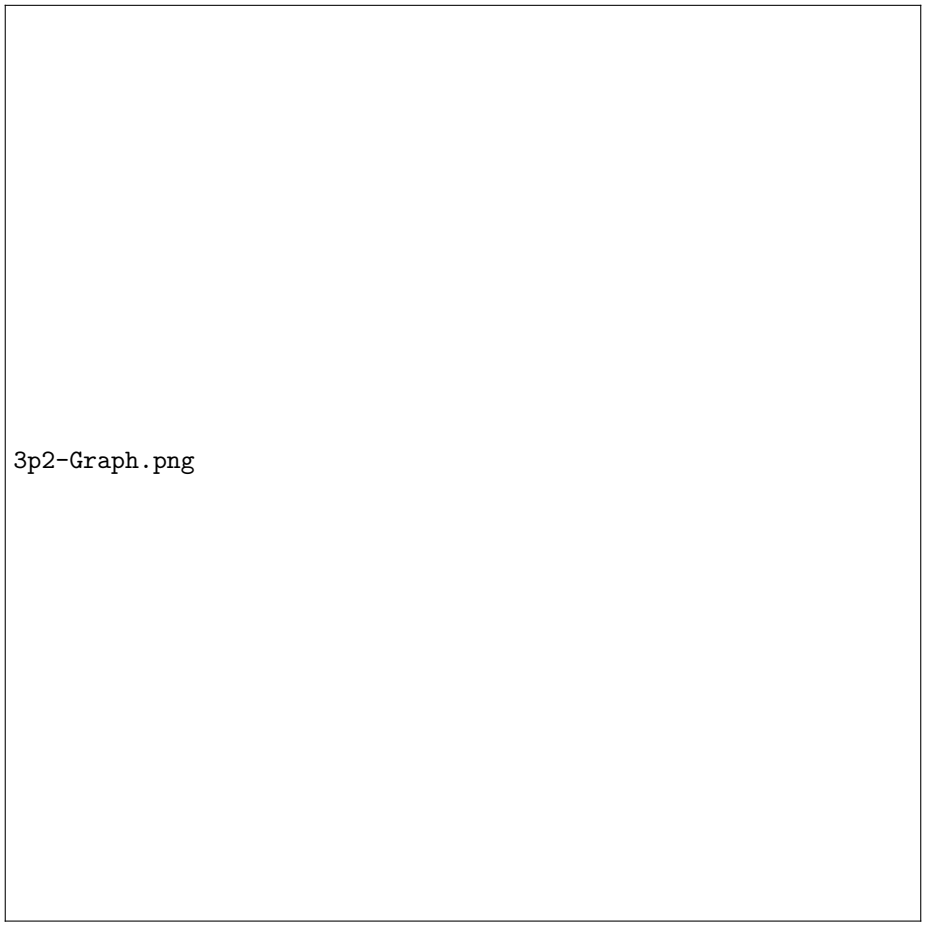
Greedy3.cpp: Our third approach implements the idea described above and is based on heuristic 1. During input processing, we associate a degree with each vertex and each rectangle. We then insert pairs of (rectangleID, rectangleDegree) into a priority queue and repeatedly extract the top element, corresponding to the most "isolated" rectangle. For each selected rectangle, we choose the incident vertex with the highest degree (i.e., the one covering the most rectangles), place a guard there, and update the degrees of the remaining vertices and rectangles accordingly. This process is repeated until all rectangles are covered.

Greedy3v2.cpp: The second version of our third approach follows the same structure as the previous one and also implements the idea described above. The only difference is that it is based on heuristic 2, where, instead of defining the rectangle degree as the sum of the degrees of its shared vertices, it is defined

as the number of rectangles with which it is in direct contact. Apart from this change, the algorithm remains identical to the previous version and terminates once all rectangles are covered. This algorithm serves as a bridge between **Greedy3.cpp** and **Greedy4.cpp**, as it shares the same algorithmic structure as **Greedy3.cpp** while using the same heuristic as **Greedy4.cpp**.

Greedy4.cpp: Our fourth and final algorithm models the rectangle partition as a graph, where nodes (rectangles) are connected by bidirectional edges if they are in direct contact. This approach still follows the idea of heuristic 2 and works similarly to **Greedy3v2.cpp**, but with an explicit graph structure. It continues to prioritize rectangles with the lowest degree, and upon selecting one, chooses its incident vertex with the highest degree. The key difference is in the update step: after selecting a vertex, edges between rectangles incident to that vertex are preserved, while all edges between those rectangles and any other rectangles not incident to the selected vertex are removed.




3p2-Graph.png

2.2 Comparative Analysis

Inp.\Alg.	Greedy1.cpp	Greedy2.cpp	Greedy3.cpp	Greedy3v2.cpp	Greedy4.cpp	Optimal (OR-Tools)
input-1-5	3	2	2	2	2	2
input-1-10	4	4	4	4	4	4
input-1-15	7	6	6	6	6	6
input-1-30	13	12	12	12	12	12
input-1-50	20	20	19	19	19	18
input-1-100	43	41	36	36	36	36
input-1-250	100	101	90	90	90	89
input-1-500	197	197	182	182	182	175
input-1-1000	397	393	357	357	357	343
input-1-5000	1967	1972	1799	1799	1799	1729
input-1-50000	19794	19874	18040	18045	18045	17352

Note that, for each input-x-y, we consider the entire set of rectangles in the partition (100%), rather than a random subset. It is important to note that, x denotes the number of instances and y the number of rectangles in the partition. These input files are same included in the provided zip file.

Regarding our first two algorithmic approaches, namely **Greedy1.cpp** and **Greedy2.cpp**, they produce similar results across all instances, with the most noticeable deviation occurring for inputs with 50,000 or more rectangles, where **Greedy1.cpp** yields slightly better results.

Regarding the greedy algorithms that prioritize "isolated" rectangles, namely **Greedy3.cpp**, **Greedy3v2.cpp**, and **Greedy4.cpp**, they all produce better results than those obtained by the first two algorithmic approaches and closer to the optimal solution across all instances. They also consistently produce the same results between themselves. This was not unexpected, since they all share the same underlying principle, differing only in their heuristic definitions and implementation details. For the 50,000 rectangle input, there is a slight variation between **Greedy3.cpp** and **Greedy3v2.cpp** / **Greedy4.cpp**, which we attribute to minor implementation nuances rather than any actual improvement. Overall, these algorithms remain closely aligned in terms of performance and stay relatively close to the optimal solution obtained via our IP OR-Tools implementation, with the gap becoming more noticeable for larger instances (50,000+ rectangles). Still, from 500+ rectangles onwards, where deviations begin to become more evident, these three algorithms continue to exhibit very similar behaviour, consistently achieving an additional value of approximately 4% compared to the optimal solution, computed as:

$$\frac{\text{GreedySolution} - \text{OptimalSolution}}{\text{OptimalSolution}} \times 100$$

Overall, the experimental results show a clear distinction between the different families of greedy approaches. The first two algorithms, **Greedy1.cpp** and **Greedy2.cpp**, produce similar results across all instances, with only minor differences appearing for larger inputs. In contrast, the approaches based on prioritizing "isolated" rectangles (**Greedy3.cpp**, **Greedy3v2.cpp**, and **Greedy4.cpp**) behave in a more consistent manner, both internally and across instances, and generally achieve better performance relative to the first two methods.

This shows that shifting the focus from vertex-based heuristics to rectangle-based prioritization has a positive impact on solution quality and that in the second family the heuristic approach doesn't show much of a difference. In particular,

However, despite these improvements, all greedy approaches remain consistently below the optimal solution obtained via our IP OR-Tools model. The performance gap becomes more pronounced as the size of the input increases, highlighting the limitations of greedy strategies for larger instances.

Chapter 3

Integer Programming

In this chapter, an Integer programming model is presented. The initial step in this process is to establish the mathematical formulation of the problem, which includes the definition of the decision variables, constraints, and objective function. Then, the implementation of this approach using Google OR-Tools is described, followed by an interpretation of the results.

3.1 Mathematical Model

Let m be the number of rectangles to cover, n the number of vertices and p_{ij} represented as follows:

$$p_{ij} = \begin{cases} 1, & \text{if rectangle } j \text{ is incident to vertex } i \\ 0, & \text{otherwise} \end{cases}$$

Decision Variables: v_i indicates wheter vertex i has a guard or not.

The objective is to minimize the total number of guards:

$$\sum_{i=1}^{n} v_i$$

subject to the following constraints:

$$\begin{cases} \sum_{i=1}^{n} v_i \times p_{ij} \geq 1, \quad \forall j = 1, 2, \ldots, m \\ v_i \in \{0, 1\}, \quad \forall i = 1, 2, \ldots, n \end{cases}$$

The objective of this constraint is to ensure that each rectangle is covered by ensuring that at least one of its vertices possesses a guard.

3.2 OR-Tools

The implementation of the Integer Programming model utilizes the `MPSolver` interface from Google OR-Tools, specifically configured to use the SCIP (Solving Constraint Integer Programs) backend.

The implementation translates the mathematical formulation directly into C++ code:

- **Variables:** Using solver→MakeIntVar(0.0, 1.0, varName), boolean decision variables x_i are instantiated for each candidate vertex in the problem space.
- **Constraints:** For each rectangle, a row constraint is created via solver→ MakeRowConstraint(1.0, inf). The variables corresponding to the vertices incident to that rectangle are assigned a coefficient of 1. This enforces the condition $\sum_{i=1}^{n} x_i \times p_{ij} \geq 1$.
- **Objective Function:** The objective function is defined by setting the coefficient of every vertex variable to 1 via objective→SetCoefficient(x[i], 1) and invoking objective→SetMinimization() to ensure the solver finds the minimum set of guards.

Additionally, to improve solver performance by reducing the search space, a theoretical lower bound for the number of guards needed is calculated. Specifically, a bound of $\lceil m/3 \rceil - 1$ (where m is the number of rectangles to be covered) is computed and added as a global row constraint via solver→MakeRowConstraint (solutionLowerbound - 1, inf). This prevents the solver from evaluating unfeasible sub-optimal branches.

Chapter 4

Constraint Programming

In this chapter, a Constraint programming model is presented. The initial step in this process is to establish the Constraint Optimization Problem (COP). Then, we discuss how to solve the COP using two different methods: maintaining arc-consistency with AC-3 and using OR-Tools.

4.1 Constraint Optimization Problem

A Constraint Optimization Problem is a tuple (X, D, C) where:

- $X = \{x_1, x_2, \ldots, x_n\}$ is the set of variables;
- $D = \{d_1, d_2, \ldots, d_n\}$ is the set of domains, where d_i is a finite set of possible values for x_i;
- $C = \{c_1, c_2, \ldots, c_m\}$ is the set of constraints;
- f is the optimization function.

The following COP can be used to model this problem:

- $X = \{v_1, v_2, \ldots, v_n\}$: set of vertices;
- $D = \{d_1, d_2, \ldots, d_n\} : d_1 = d_2 = \cdots = d_n = \{0, 1\}$;
- C: $\sum_{i=1}^{n} v_i \times p_{ij} \geq 1 \in C, \quad \forall j = 1, 2, \ldots, m$;
- $f = \sum_{i=1}^{n} v_i$.

The model under discussion is essentially analogous to the model for integer programming, but expressed in the form of a COP.

4.2 AC-3

In this section, the methodology for solving the COP mentioned in the previous section, by maintaining arc consistency, will be demonstrated using AC-3 as a mean of filtering and propagating information.

Two different approaches were considered with regard to the AC-3. Initially, it seemed a good idea to perform a dual graph translation to obtain the corresponding binary COP. However, this would result in exponential complexity for the translation and a combinatorial explosion in the number of constraints in the resulting binary formulation. Therefore, we opted for a generalised version of the AC-3, which, although still computationally expensive, does not require an initial translation step.

The following pseudocode describes the AC-3 algorithm:

procedure AC3(X, D, C)
  $Q \leftarrow \{(i, c) \mid c \in C,\ x_i \in \text{scope}(c)\}$
  while $Q \neq \emptyset$ **do**
    $(i, c) \leftarrow \text{Fetch}(Q)$
    if REVISE(i, c) **then**
      $Q \leftarrow Q \cup \{(j, c') \mid c' \in C,\ c' \neq c,\ j \neq i,\ \{x_i, x_j\} \subseteq \text{scope}(c')\}$
    end if
  end while
end procedure

function REVISE(i, c)
  change $\leftarrow false$
  for each $a \in d_i$ **do**
    if $\forall a_1 \in d_1, \ldots, a_{i-1} \in d_{i-1}, a_{i+1} \in d_{i+1}, \ldots, a_k \in d_k, \neg c(x_1 \leftarrow a_1, \ldots, x_i \leftarrow a, \ldots, x_k \leftarrow a_k)$ **then**
      remove a from d_i
      change $\leftarrow true$
    end if
  end for
  return *change*
end function

Although AC-3 itself is employed for the filtration of information, a method of propagating the filtered results is still required. The search algorithm that has been selected to work in conjunction with AC-3 is known as backtracking.

The following pseudocode describes the backtracking algorithm:

```
function BT(τ, X, D, C)
    if X is empty then
        return true
    end if
    if ¬AC3(X, D, C) then
        return false
    end if
    x_i ← Select(X)
    for each a ∈ Domain(x_i) do
        τ(x_i) ← a
        if consistent(τ, C, x_i) then
            if BT(τ, X, D[d_i → {a}], C) then
                return true
            end if
        end if
        restore state or use persistent data structures
    end for
    return false
end function
```

```
function CONSISTENT(τ, C, x_i)
    for each c ∈ C s.t scope(c) ⊆ vars(τ) and x_i ∈ scope(c) do
        if ¬c(τ) then
            return false
        end if
    end for
    return true
end function
```

As AC-3 and backtracking only determine the existence of a feasible solution within a given bound, an additional optimisation strategy is required to minimise the objective function. Thefore, we perform a binary search over the objective value. Let m be the upper bound for the number of selected vertices. A new constraint is temporarily added to the CPO.

If no solution is found, the lower bound variable is updated. Otherwise, a feasible solution has been found and the upper bound variable can be reduced. This process continues until the optimal value is identified. At the end, the optimal value is stored in the lower bound variable l.

```
procedure BINARYSEARCH(X, D, C)
    l ← 0
    r ← m
    while l < r do
        m ← l + (r − l)/2
```

add constraint $f \leq m$
if $\mathrm{BT}(\tau, X, D, C)$ **then**
$r \leftarrow m$
else
$l \leftarrow m + 1$
end if
end while
return l
end procedure

4.3 OR-Tools

To implement the Constraint Optimization Problem, the OR-Tools CP-SAT solver is employed via the `CpModelBuilder` API. The CP-SAT solver utilizes satisfiability (SAT) methods under the hood to efficiently explore the domain.

In accordance with the base problem formulation, the model construction focuses solely on the optimal placement of guards:

- **Variables:** The decision variables v_i are initialized using cp_model.NewIntVar(0, 1, name), strictly bounding the domain of each candidate vertex to $\{0, 1\}$.
- **Constraints:** The coverage constraints are constructed using linear expressions (`LinearExpr`). For each rectangle, the boolean variables of its incident vertices are summed. The constraint is then strictly enforced using cp_model.AddGreaterOrEqual(expr, 1), ensuring every rectangle is covered by at least one guard.
- **Objective:** The objective is defined as minimizing the total number of placed guards. This is achieved by summing all v_i variables and applying cp_model.Minimize(expr).

The model is compiled and solved using `Solve(cp_model.Build())`. The solver returns a `CpSolverResponse`, which yields the optimal geometric configuration of guards that minimizes the total count.

Chapter 5

Extensions

In this chapter, two extensions of the original problem are presented.

5.1 Coloring

Given the minimum number of guards required to cover all rectangles, the objective is to assign colours to the guards such that guards covering the same rectangle have different colours, while minimising the total number of colours used.

This problem belongs to the class of multi-objective optimisation problems, since the objective is to minimise both the number of guards used and the number of colours used. This finding suggests that one minimization is considerably more significant than the other. It is therefore proposed that a weighted-sum method be employed. However, following a review of the relevant online literature, it became apparent that proving the veracity of our model and defining a weight for each constraint would be a challenge. Consequently, a sequential optimisation method was employed, whereby multiple objectives were transformed into single-objective sub-problems. The first objective was the minimisation of the guards, and the second was the minimisation of the colours, with an additional restriction that the number of guards used must be equal to the minimum found in the previous model.

The first optimisation problem, concerning the minimisation of the number of guards, was presented in the previous chapters. Consequently, the present chapter is exclusively concerned with the second optimisation problem.

Let m be the number of rectangles to cover, n the number of vertices, s the number of guards to use and z be the maximum number of colors, that is, n. Let

$$p_{ij} = \begin{cases} 1, & \text{if rectangle } j \text{ is incident to vertex } i \\ 0, & \text{otherwise} \end{cases}$$

The decision variables are defined as follows:

- v_i indicates if vertex i contains a guard;
- u_{ij} indicates if vertex i has color j;
- c_j indicates if color j is used.

The objective is to minimize the number of guard types used:

$$\min \sum_{i=1}^{z} c_i$$

subject to the following constraints:

$$\begin{cases} \sum_{i=1}^{n} v_i \times p_{ij} \geq 1, \quad \forall j = 1, 2, \ldots, m \\ \sum_{i=1}^{n} v_i = s \\ \sum_{j=1}^{z} u_{ij} = v_i, \quad \forall i = 1, 2, \ldots, n \\ \sum_{i=1}^{n} u_{ij} \geq c_j, \quad \forall j = 1, 2, \ldots, z \\ \sum_{i=1}^{n} p_{ir} \times u_{ik} \leq 1, \quad \forall r = 1, 2, \ldots, m, \ \forall k = 1, 2, \ldots, z \\ u_{ij} \leq c_j, \quad \forall i = 1, 2, \ldots, n, \ \forall j = 1, 2, \ldots, z \\ v_i \in \{0, 1\}, \quad \forall i = 1, 2, \ldots, n \\ u_{ij} \in \{0, 1\}, \quad \forall i = 1, 2, \ldots, n, \ \forall j = 1, 2, \ldots, z \\ c_i \in \{0, 1\}, \quad \forall i = 1, 2, \ldots, z \end{cases}$$

The first constraint guarantees that every rectangle is covered. The second constraint ensures that exactly s guards are used. The third constraint associates each selected vertex with only one color. The fourth and sixth constraints establish if a color is used or not. Finally, the fifth constraint ensures that no rectangle is covered by more than one guard of the same color.

5.2 Extended Visibility

This extension is highly similar to the original problem, yet a guard may cover adjacent rectangles up to a distance D.

Let m be the number of rectangles to cover, n the number of vertices, D the maximum distance and d_{ij} the minimum distance for vertex i to rectangle j. Let

$$p_{ij} = \begin{cases} 1, & \text{if } d_{ij} \leq D \\ 0, & \text{otherwise} \end{cases}$$

d_{ij} can easily be computed as follows:

For each vertex i, the search is initialized with all rectangles incident to i, which form the rectangles at distance 0. Subsequently, a BFS is performed over the rectangle adjacency graph, giving us the minimum distance from vertex i to every rectangle j.

The decision variables are defined as follows:

- v_i indicates if vertex i contains a guard;

The objective is to minimize the total number of guards:

$$\sum_{i=1}^{n} v_i$$

subject to the following constraints:

$$\begin{cases} \sum_{i=1}^{n} v_i \times p_{ij} \geq 1, & \forall j = 1, 2, \ldots, m \\ v_i \in \{0, 1\}, & \forall i = 1, 2, \ldots, n \end{cases}$$

Chapter 6

Conclusion